

interesting applications of searching, including inverting mathematical functions.

Insertion Sort

Imagine that you've been asked to sort all the files in a filing cabinet. Each file has a number assigned to it, and you need to rearrange the files so that the file with the lowest number is first in the cabinet, the file with the highest number is last, and the files' numbers proceed in order in between.

Whatever method you follow as you sort the filing cabinet, we can describe it as a "sorting algorithm." But before you even think of opening Python to code an algorithm for this, take a moment to pause and consider how you would sort such a filing cabinet in real life. This may seem like a mundane task, but allow the adventurer within you to creatively consider a broad range of possibilities.

In this section, we present a very simple sorting algorithm called *insertion sort*. This method relies on looking at each item in a list one at a time and inserting it into a new list that ends up being correctly sorted. Our algorithm's code will have two sections: an insertion section, which performs the humble task of inserting a file into a list, and a sorting section, which performs insertion repeatedly until we have completed our sorting task.

Putting the Insertion in Insertion Sort

First, consider the task of insertion itself. Imagine that you have a filing cabinet whose files are already perfectly sorted. If someone hands you one new file and asks you to insert it into the right (sorted) position in the filing cabinet, how do you accomplish that? The task may seem so simple that it doesn't warrant an explanation, or even the possibility of one (*just do it!* you might think). But in the world of algorithms, every task, however humble, must be explained completely.

The following method describes a reasonable algorithm for inserting one file into a sorted filing cabinet. We'll call the file we need to insert the "file to insert." We'll say that we can compare two files and call one file "higher